
Workshop ROSCON

De 0 à utiliser un UR10 avec ROS 2

ROSCON FR 2026

21/06/2026



LE FUTUR
DE VOS USINES



ROS et ROS 2, une introduction

ROS – L'Histoire

► Arrêter de réinventer la roue

- Créé par la startup Willow Garage en 2007
- Objectif : Fournir une base logicielle robotique commune et open source pour la recherche, basée sur
 - Réutilisation des données facilitée
 - Transfert et déploiement simplifié
- 2010 -  ROS
- 2017 - 

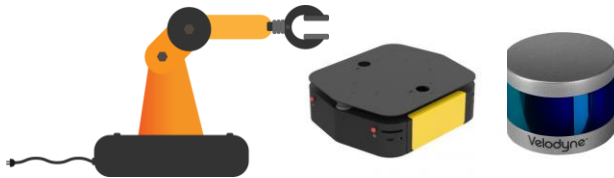
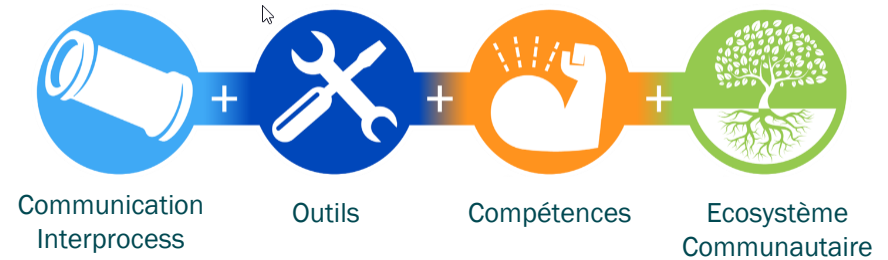
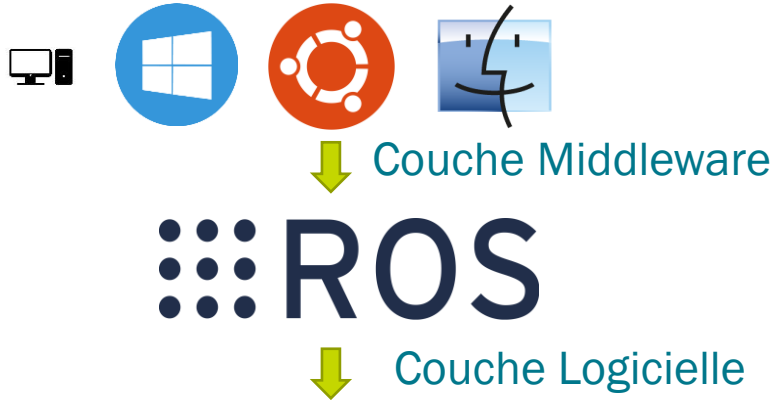
Actuellement maintenu par la société

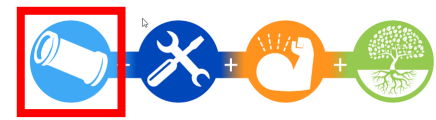


Source : <https://www.robotshop.com/community/blog/show/robot-humor-dinner-comics>

ROS (Robot Operating System) – Les concepts

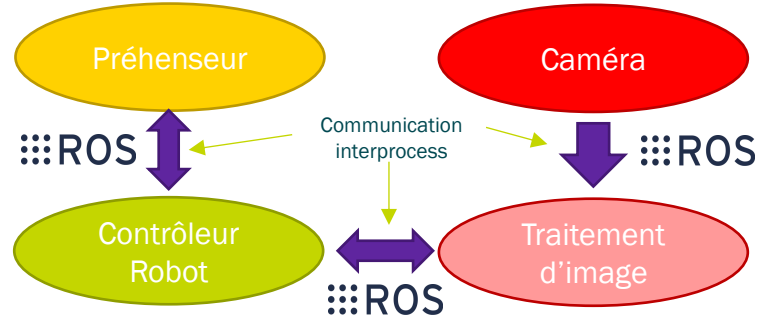
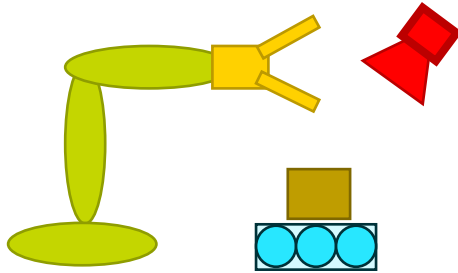
► Un middleware avec 4 aspects principaux



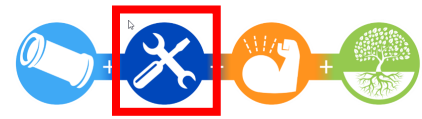


Communication Inter-process

► Cas Pick and Place

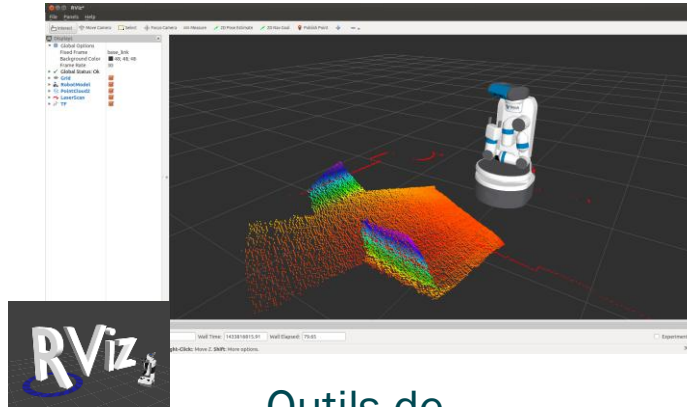


ROS fournit un protocole d'échange d'informations « standardisé » entre les différents modules basé sur la publication et la souscription.



Des outils logiciels

► Accélère le développement

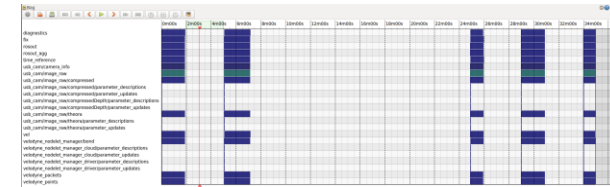


Outils de
visualisation

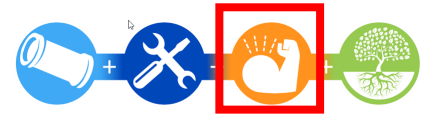


GAZEBO

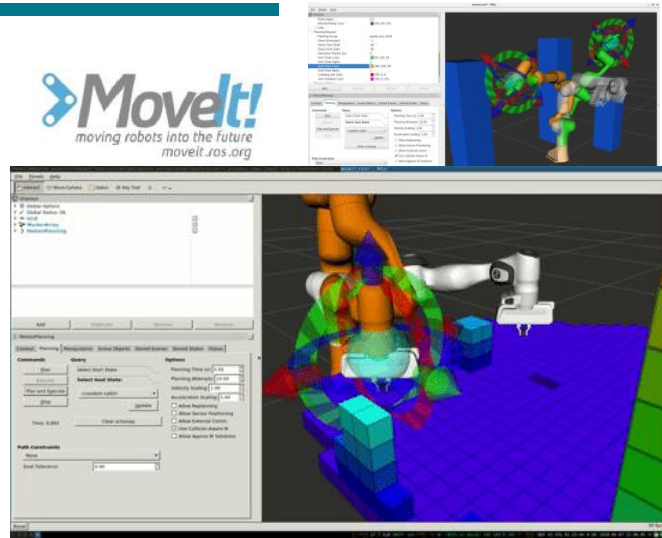
Outils de
simulation



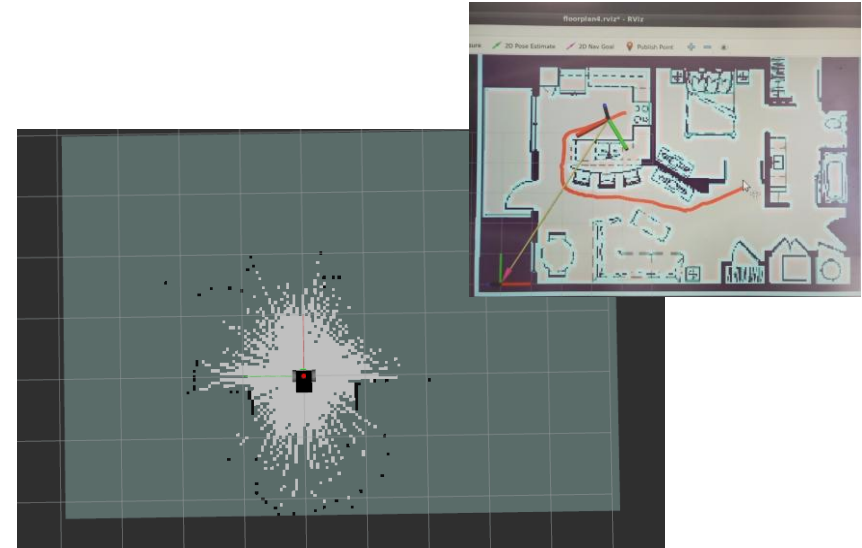
Enregistrement et
relecture de données



Algorithmes et compétences gratuites « sur étagère »



Planificateur de trajectoires
(avec évitement d'obstacle)

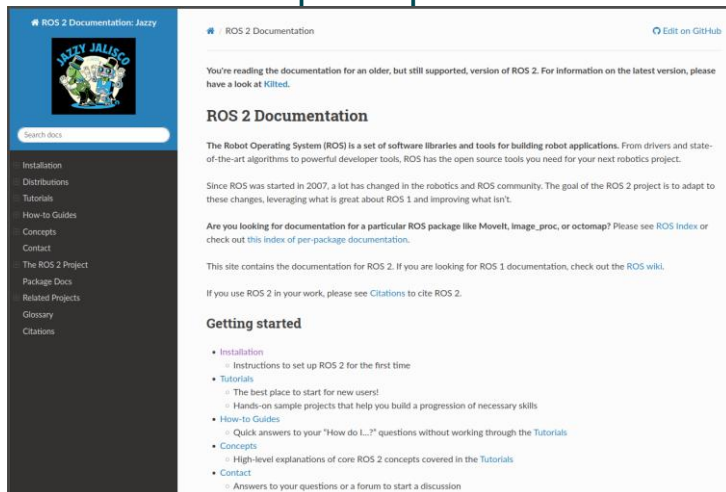


Outils de cartographie et de navigation 2D

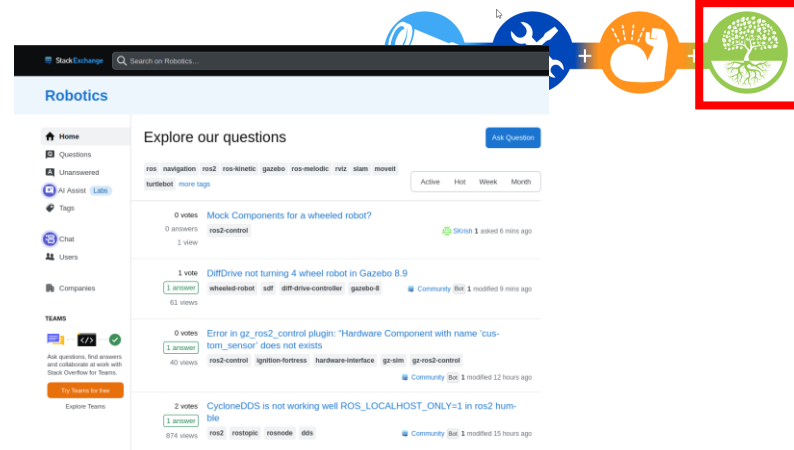
Une grande communauté

ROS.org

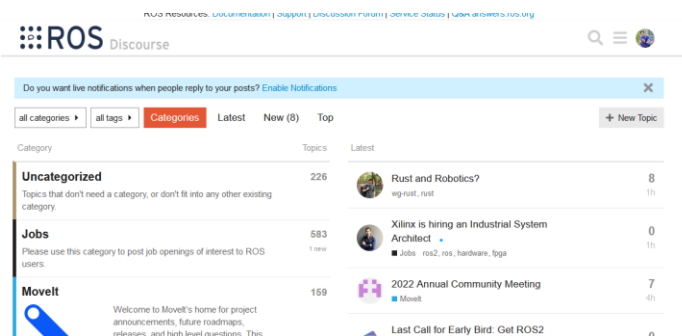
Site principal



<https://docs.ros.org/en/jazzy/index.html>



Questions-réponses (Robotics Stackexchange)



Forum de la communauté (Discourse)

Pourquoi ROS ?

- Profiter d'algorithmes open source à l'état de l'art
- Faciliter l'intégration avec différents matériels
- Partager les données facilement

« Les robots et les défis robotiques sont tellement complexes qu'ils deviennent impossibles à relever seul »

O.Stasse, directeur de l'équipe GEPETO du CNRS LAAS,

Journée Francophone ROS 2017

Pourquoi ROS 2 ?

► Des difficultés à poursuivre les développements de ROS 1

ROS n'est pas taillé pour des évolutions majeures :

1. Architecture « datée » et une maintenance lourde (deux fois le travail pour maintenir la compatibilité cpp / python)
2. Maintenance d'éléments « externe-interne » (actionlib, certains messages, ...) qui a explosé
3. Retard technologique (sécurité, protocole de communication ...)

Deux choix :

- ⇒ Refonte de ROS 1, apportant de grosses incompatibilités
- ⇒ Refaire de 0 => ROS 2

Introduction à ROS

► Questions ?





**Avez-vous installé
ROS 2 ?**



Les concepts de base de ROS 2

Les topics

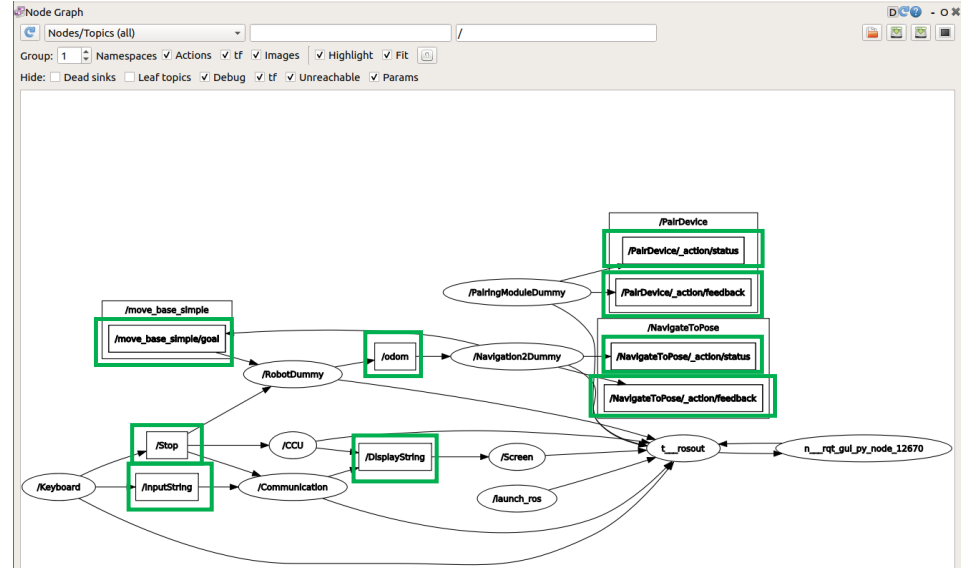
► La base de la communication

Les **topics** – Ce sont des « lieux d'échanges » de données. Ils sont nommés.

Il connecte toujours :

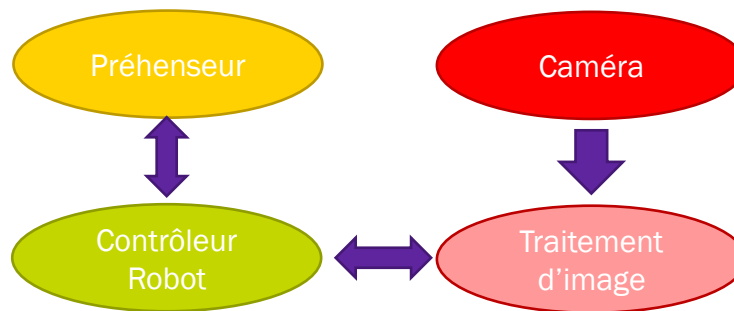
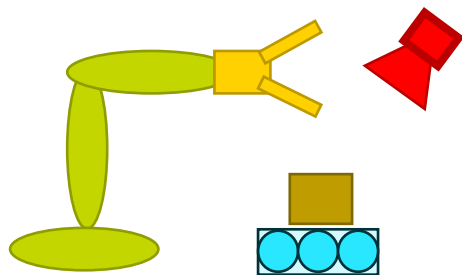
- Un (ou plusieurs) nœud(s) fournisseur de données (**Publisher**)
- Un ou plusieurs nœuds consommateurs de données (**Subscriber**)

Le **Publisher** et le **Subscriber** doivent s'être accordés sur la structure des données échangées (**Message**)



Exemple

► Pick and place

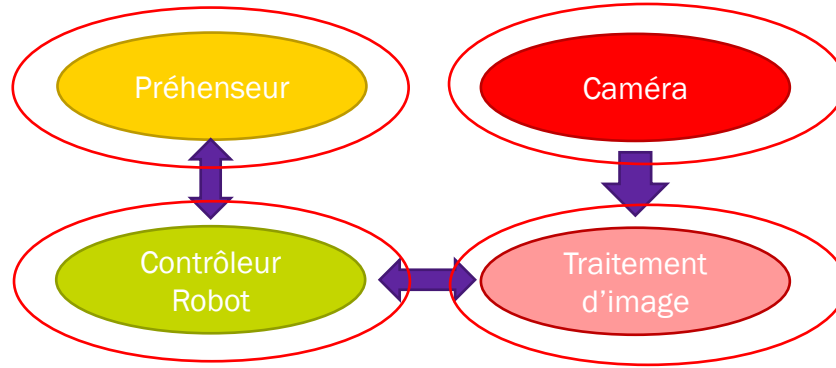
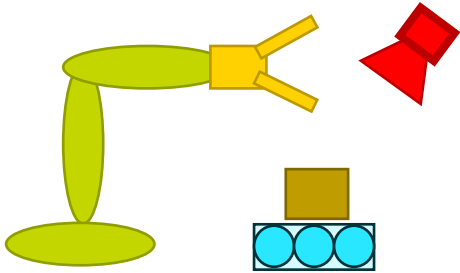


Trouvé les nœuds ?

Trouvé les topics ?

Exemple

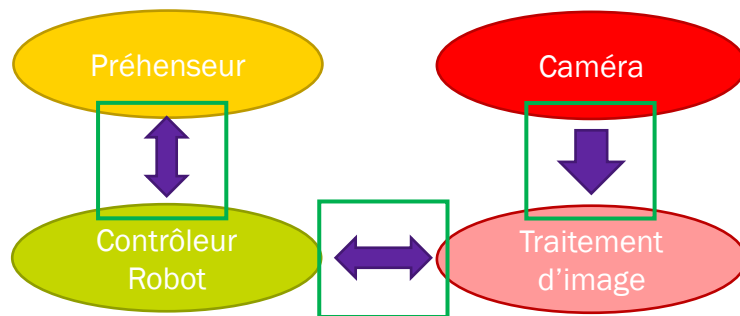
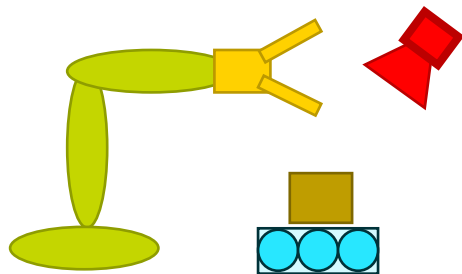
► Pick and place



Les nœuds sont les éléments qui communiquent ensemble

Exemple

► Pick and place



Les topics sont les lieux d'échanges des données

Les échanges de données sur ROS

► 3 types d'échanges principaux

1. Messages
2. Services
3. Actions

(Paramètres)

Les topics et les messages

► Échange asynchrone et unilatéral

Il est possible d'avoir plusieurs subscriber pour chaque topic

Il est possible d'avoir plusieurs publisher pour chaque topic (**mais attention, mauvaise pratique!**)

Fonctionne de manière asynchrone

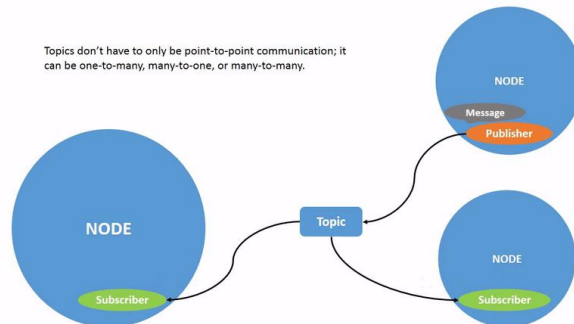
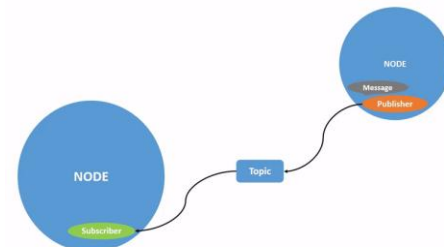
Pas besoin d'attendre que le subscriber est lu le message pour envoyer un nouveau message

Possibilité d'avoir une mémoire/buffer (Queue) côté publisher

Chaque topic contient une structure de données appelée Message

Il existe des messages « standards »

Il est possible de créer des messages « custom »



Topics don't have to only be point-to-point communication; it can be one-to-many, many-to-one, or many-to-many.

Les messages

► Une structure de données

Défini dans un fichier .msg

- Peut contenir plusieurs champs « simple »
- Peut-être composé d'autres messages

Il existe des messages « communs » et des messages « standards »

- http://wiki.ros.org/common_msgs => Liés à leur utilisation (Pose, Image, ...)
- http://wiki.ros.org/std_msgs => Format de données « standards » (Booléen, Tableau de float64, ...)

[geometry_msgs/Point Message](#)

File: `geometry_msgs/Point.msg`

Raw Message Definition

```
# This contains the position of a point in free space
float64 x
float64 y
float64 z
```

Compact Message Definition

```
float64 x
float64 y
float64 z
```

[geometry_msgs/Pose Message](#)

File: `geometry_msgs/Pose.msg`

Raw Message Definition

```
# A representation of pose in free space, composed of position
Point position
Quaternion orientation
```

Compact Message Definition

```
geometry_msgs/Point position
geometry_msgs/Quaternion orientation
```

Démonstration Live

- **Publisher**
- **Subscriber**
- **Subscriber 2**
- **Lancement Publisher 2**
- **Graph**
- **Commande ROS 2 TOPIC**

Les services

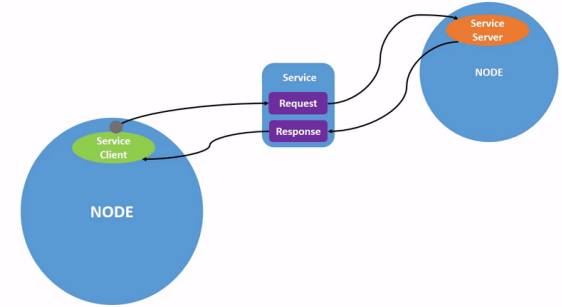
► Communication synchrone

Fonctionnement sur la base de Requête (Request) et Réponse (Response)

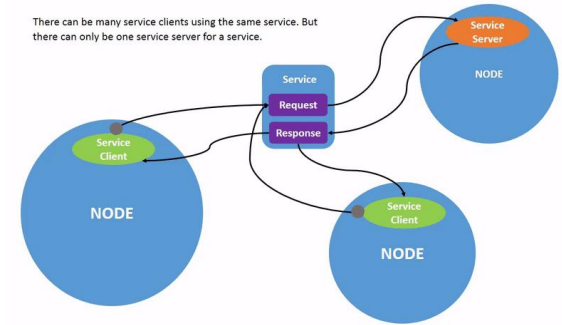
- Le serveur attend une requête (qui peut contenir des données) et renvoie une réponse (qui peut contenir des données)
- Échange ponctuel. La connexion est, en générale, recrée à chaque appel.

Un service est nommé et est lié à une structure de données double (Requête et Réponse)

Échanges à court terme avec des besoins de synchronisations sur les échanges



There can be many service clients using the same service. But there can only be one service server for a service.



La structure des services (.srv)

► Structure de données double

Défini dans un fichier .srv

- Même construction de donnée qu'un .msg (donnée standard, ou d'autres structures de messages)
- Contient à la fois, la structure de la requête et la structure de la réponse séparée par un « --- »

Il existe très peu de services standards, mais beaucoup d'algorithmes et d'outils ROS fournissent des services

std_srvs/SetBool Service

File: `std_srvs/SetBool.srv`

Raw Message Definition

```
bool data # e.g. for hardware enabling / disabling
---
bool success # indicate successful run of triggered service
string message # informational, e.g. for error message
```

Compact Message Definition

```
bool data
bool success
string message
```

Structure
Requête

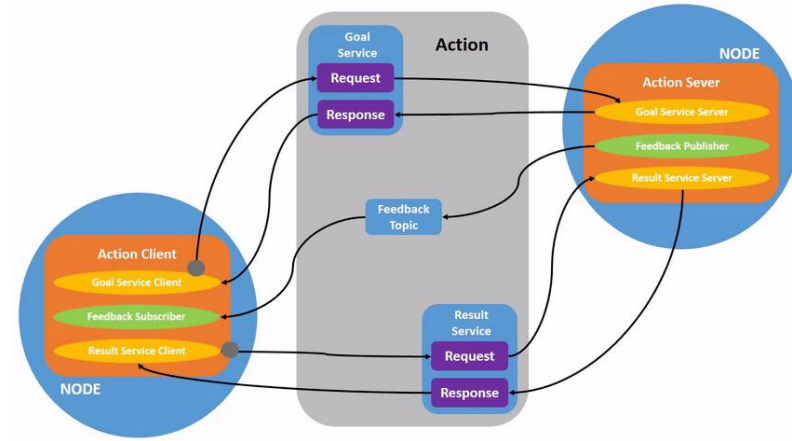
Structure
Réponse

Les actions

► Communication annulable synchrone avec feedback

Fonctionnement complexe basé sur des services et des topics synchronisés par une machine à état interne.

- Le serveur attend une requête (Goal) (qui peut contenir des données) et renvoie des données standards (acceptation ou refus)
- Le serveur exécute l'action en publiant (généralement) des messages Feedback sur un topic
- Le client peut à tout moment annuler l'action (Cancel)
- Une fois l'exécution terminée, le serveur renvoie une réponse (Result) qui contient des données et le statut (Succeeded, Aborted, Canceled)



Échange à long terme prévu pour des actions longues

La gestion des fichiers dans ROS

► Les grands concepts

ROS fonctionne grandement avec des variables d'environnement pour trouver et faciliter l'accès aux différents éléments (bibliothèque, exécutable)

ROS utilise un système de compilation configurable appelé Ament, lui-même appelé par un outil appelé COLCON

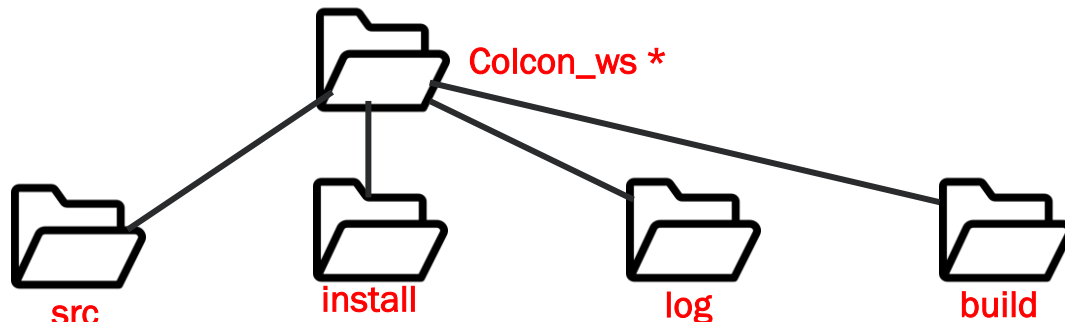
Les concepts importants pour la gestion des fichiers :

- Les workspaces
- Les packages
- Les manifests

Les workspace

► La racine des dossiers ROS

Un workspace est l'endroit où sont stockés et organisés les fichiers ROS



Colcon_ws * - Dossier d'origine du workspace (peut avoir n'importe quel nom)

Src – Dossier contenant les fichiers sources organisés sous forme de packages (à créer à la main !)

Install – Dossier généré automatiquement par colcon qui contient les exécutables et les bibliothèques compilés ou copiés (si pas compiler). Il contient également les scripts pour les variables d'environnement pour utiliser le workspace

Log – Dossier généré automatiquement par colcon contenant les logs des opérations de compilations

Build – Dossier généré automatiquement par colcon contenant les fichiers « cache » des opérations de compilations

Package

► Brique minimale dans le système de fichier de ROS

Élément minimal pour la compilation et l'exécution d'un système sous ROS.

Il peut être installé deux manières :

- Installer « déjà compilé » dans le workspace d'origine de ROS à l'aide du système de packages de l'OS (par exemple Debian pour Ubuntu)
- Compiler et installer localement depuis les sources dans un workspace créé manuellement. Les sources font alors partie du package.

L'élément minimal pour un package pour être reconnu par COLCON est le Manifest (package.xml). D'autres éléments peuvent être nécessaires pour la compilation (CMakeLists.txt pour les fichiers C++ par exemple).

Le Manifest

► Le fichier package.xml

Un fichier permettant à ROS de comprendre le contenu, les dépendances et la manière de compiler le package. C'est la carte d'identité du package.

Permet d'éviter le problème de « ça fonctionne sur mon PC » si bien rempli.

```
xml Copier le code

<package format="3">
  <name>my_package</name>
  <version>0.0.0</version>
  <description>My ROS 2 package</description>
  <maintainer email="your_email@example.com">Your Name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>roscpp</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>
</package>
```

La compilation

► Simple et efficace

Une fois à la racine d'un workspace (**!!! Important !!!**), pour compiler l'ensemble des packages contenus dans le dossier « src » d'un workspace

La commande `colcon build` permet de compiler tout le workspace.

Pour charger les variables d'environnement de ce workspace, il est nécessaires d'utiliser le script qui est généré si la compilation a abouti. Pour Ubuntu :

```
source ws_path/install/setup.bash
```

ROS 2 – Concepts et workspaces

► Questions ?





Exercice 1

Créer un workspace,
un package, et
compiler des sources

Correction

- Avez-vous des questions ?



Les outils de ROS

Les lignes de commande

► Le B-A-BA de ROS

Les lignes de commande permettent de faire énormément de choses sur ROS :

- Lancer un ou plusieurs exécutables
- Enregistrer ou relire des données échangées sur le réseau
- Faire de l'introspection sur ce qu'il se passe actuellement
- Publier / Ecouter sur des topics
- Appeler des services
- ...

Toutes les commandes ROS commencent par :

```
ros2
```

Exemple

```
ros2 --help
```

Les lignes de commandes

► Lancer un programme

- Exécute un programme du package (doit être exécutable)

```
ros2 run package_name nom_du_programme arguments
```

- Lance le fichier launch (et les programmes associés)
 - Possibilité de passer les arguments déclarés (utiliser « := »)

```
ros2 launch package_name fichier_launch
```

Les lignes de commandes

► Les topics

```
ros2 topic info nom_du_topic -v
```

- Donne toutes les informations sur le topic (publishers, subscribers, type, QoS)

```
ros2 topic echo nom_du_topic
```

- Souscrit au topic et affiche le contenu des messages reçus

```
ros2 topic pub nom_du_topic type_de_message "{contenu}" -r X
```

- Crée un publisher sur le topic qui publie a une fréquence de *X* le contenu
 - Il faut mettre le contenu sous forme de dictionnaire python.

Les lignes de commandes

► Les topics

```
ros2 topic list
```

- Donne tous les topics présents sur le réseau

```
ros2 topic hz nom_du_topic
```

- Souscrit au topic et affiche la fréquence des messages reçus

Les lignes de commandes

► Les services

```
ros2 service info nom_du_service
```

- Donne toutes les informations sur le service (nœud serveur, type)

```
ros2 service list
```

- Liste les services actifs

```
ros2 service call nom_du_service type_de_service "contenu"
```

- Fait une requête sur le service et affiche la réponse
 - Contenue sous forme de dictionnaire python

Les lignes de commandes

► Les noeuds

```
ros2 node info nom_du_noeud
```

- Donne toutes les informations sur le noeud (service server, publishers, subscriber, IP, PID)

```
ros2 node list
```

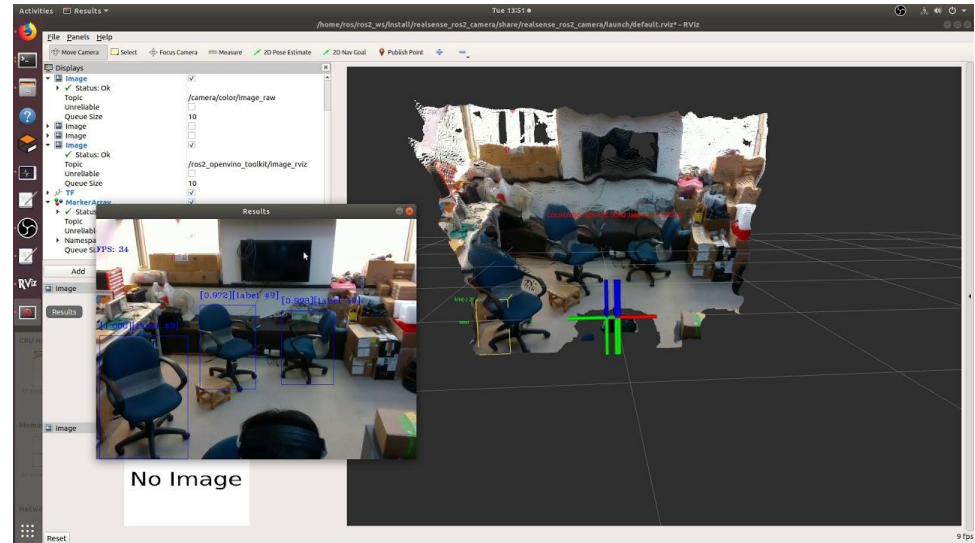
- Liste les nœuds présents sur le réseau

RVIZ 2

► Outil de visualisation

rviz2

- Contient des visualisations préparées
 - Modèle 3D d'un robot
 - Repères géométriques
 - Nuages de points
 - Images
 -
- Possibilité d'éditer complètement le layout
- Possibilité d'intégrer dans une IHM
https://github.com/mjeronimo/rviz_embed_test



RQT

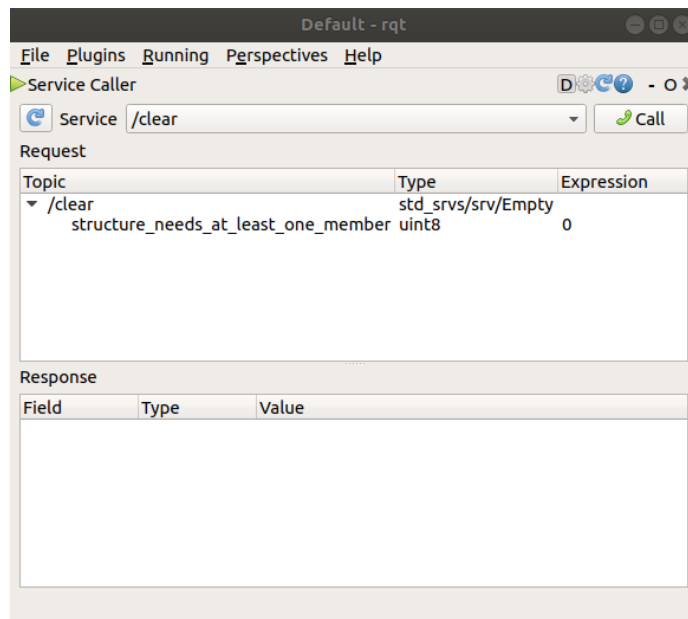
► L'outil de debug ultime -

<https://docs.ros.org/en/humble/Concepts/Intermediate/About-RQt.html>

rqt

- **Contient beaucoup d'éléments utiles**

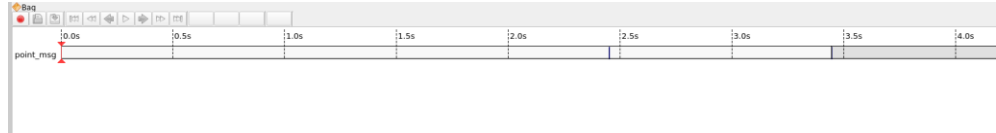
- Loggers
- Node Graph
- Publisher
- Service Caller
- ...



ROSBag

► Enregistrement des topics

rqt_bag



Permet d'enregistrer le contenu des messages, leurs timing et de les rejouer

Outil très utile pour le logging / debugging à distance / tests

Démonstration

- CLI
- RVIZ 2
- RQT

Outils

► Questions ?





Exercice 2

Utiliser les outils ROS

Correction

- Avez-vous des questions ?



Manipulation avec ROS

Les bases

Package Transform (TF)

► Un package pour gérer tous les repères géométriques

Problèmes classiques en robotique :

- Quelle est la matrice de passage du repère base robot au repère effecteur ?
- Quelle est la coordonnée de ce point dans le repère du robot ?
- Comment on passe d'un angle en Euler XYZ a un quaternion ?
- Où était le robot il y a 10 secondes par rapport au centre de cette image ?

➔ TF (TF2) et ses outils permettent de répondre à tous ces problèmes

Il donne un ensemble outils mathématiques (et d'APIs pour ROS) permettant de faire toutes ces opérations simplement.

Package Transform (TF)

► Des APIs particulières

Système complexe basé sur un topic « /tf » (éventuellement « /static_tf »)

- Ce système tient à jour une liste des données géométriques grâce à ce topic.
- Il est très difficile de comprendre manuellement ce qui se passe sur ce topic.
- Il ne faut pas utiliser des Publishers ou des Subscribers « classiques ».
- Heureusement, il y a déjà beaucoup de choses qui sont déjà automatisées

Si l'on souhaite ajouter / calculer une transformée géométrique, on utilise :

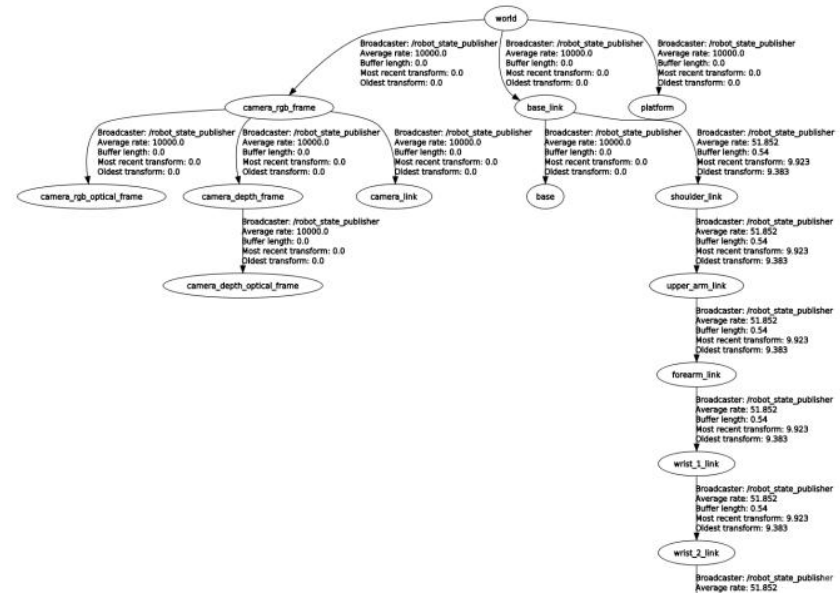
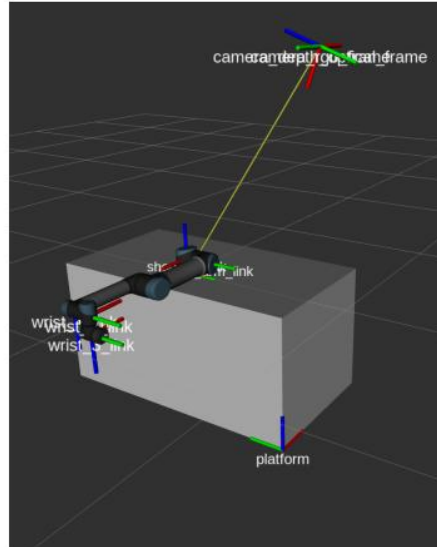
- Un broadcaster (pour envoyer une transformée géométrique)
- Un listener (pour calculer une transformée géométrique)

TF2 – Arbre de transformées

► Un outil pour savoir qui est placé par rapport à quoi

Très utiles pour déboguer

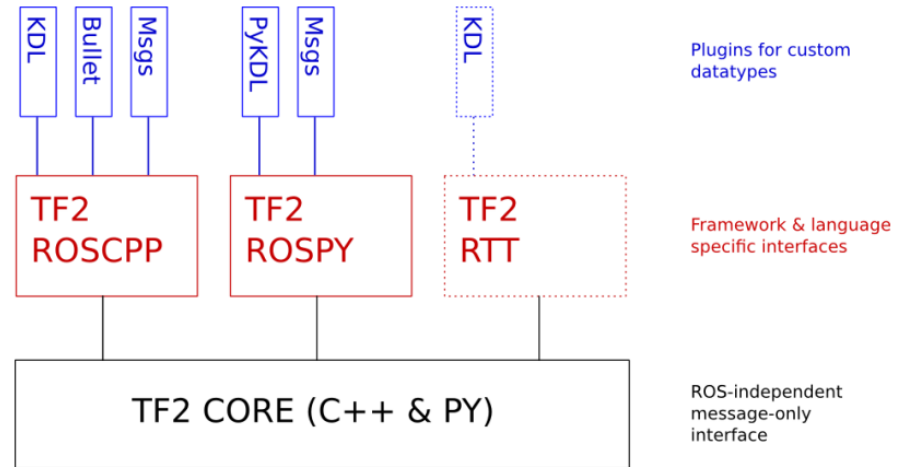
Chaines ouvertes
uniquement



TF2 - Architecture

► Indépendant de ROS

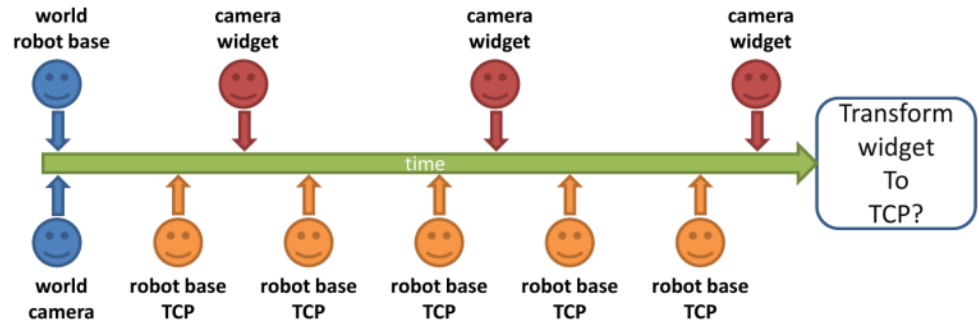
- Possibilité d'utiliser le système en dehors de ROS
- Interfaçage avec ROS automatique



TF2 – Synchronisation temporelle

► TF2 permet d'obtenir des approximations de la position dans le temps

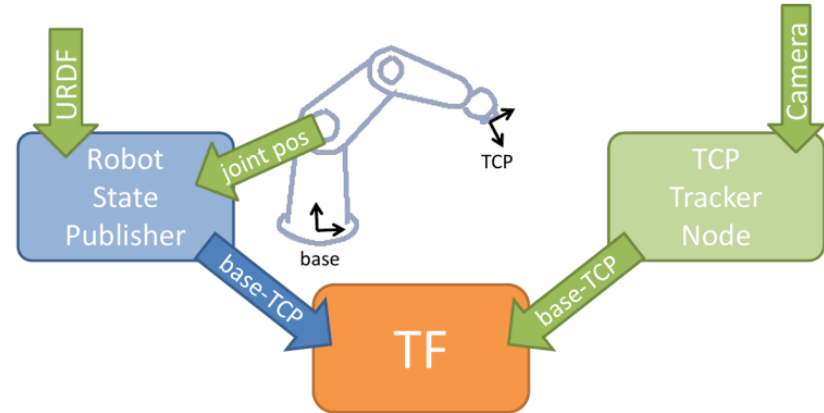
- Buffer de 10 secondes pour chaque broadcaster (configurable)
- Possibilité de définir le moment pour obtenir une transformée (la dernière en date, il y a 1 seconde)
- Possibilité de faire de l'interpolation (dans le passé)
- Fortement influencé par la fréquence de « publication »



TF2 - Broadcasters

► Les publishers de TF2

- Possibilité d'en créer à la main (via les API)
- En général, on utilise, pour un bras robot des broadcaster automatique appelés: « Robot State Publisher »
- Récupère les données articulaires du robot (topic « joint_state ») et un fichier de description géométrique (format URDF) pour automatiquement générer toute les transformées
- Il existe des broadcaster « statiques » pour envoyer des informations de transformée « fixes »



Démonstration

Lancer un exemple de broadcaster

Déplacer le robot

Faire un echo sur une position

TF2

► Questions ?

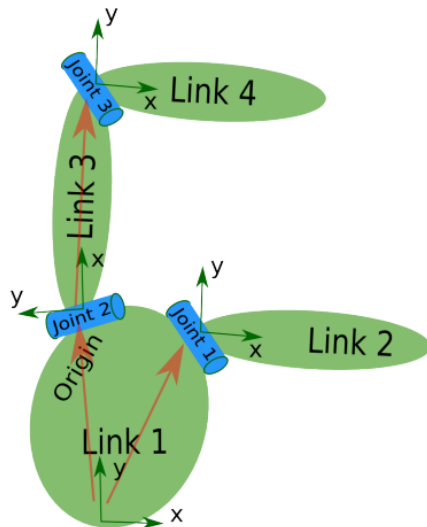




Modèle d'un robot URDF

Exemple d'URDF

- Toutes les URDFs utilisent les balises « robot »
- On utilise principalement les balises link et joint
- L'origine est toujours un link
- Un joint est toujours entre deux link (balise parent et child)



```
<robot name="test_robot">
  <link name="link1" />
  <link name="link2" />
  <link name="link3" />
  <link name="link4" />

  <joint name="joint1" type="continuous">
    <parent link="link1"/>
    <child link="link2"/>
  </joint>

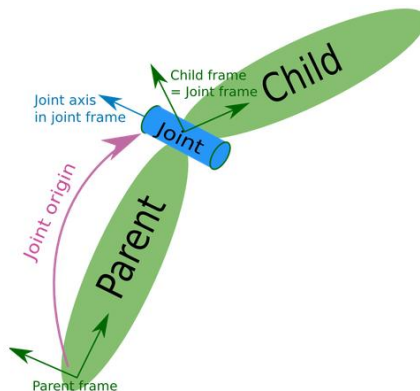
  <joint name="joint2" type="continuous">
    <parent link="link2"/>
    <child link="link3"/>
  </joint>

  <joint name="joint3" type="continuous">
    <parent link="link3"/>
    <child link="link4"/>
  </joint>
</robot>
```

Ajout des dimensions

► <http://wiki.ros.org/urdf/XML/joint>

- Exprimé dans le repère parent
- Le link enfant est exprimé dans le repère du joint
- Le joint précédent est le repère parent



```
<robot name="test_robot">
  <link name="link1" />
  <link name="link2" />
  <link name="link3" />
  <link name="link4" />

  <joint name="joint1" type="continuous">
    <parent link="link1"/>
    <child link="link2"/>
    <origin xyz="5 3 0" rpy="0 0 0" />
  </joint>

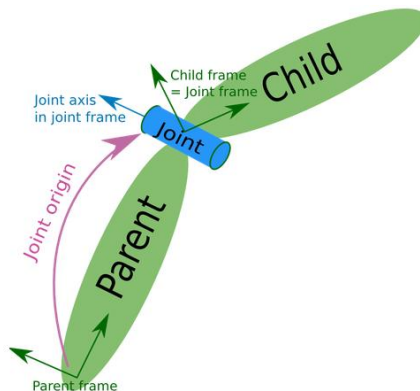
  <joint name="joint2" type="continuous">
    <parent link="link1"/>
    <child link="link3"/>
    <origin xyz="-2 5 0" rpy="0 0 1.57" />
  </joint>

  <joint name="joint3" type="continuous">
    <parent link="link3"/>
    <child link="link4"/>
    <origin xyz="5 0 0" rpy="0 0 -1.57" />
  </joint>
</robot>
```

Ajout des propriétés cinématiques

► <http://wiki.ros.org/urdf/XML/joint>

- L'axe est exprimé dans le repère du joint (après avoir été positionné dans le repère du parent)
- Possibilité de définir des limites (positions, vitesses, effort)



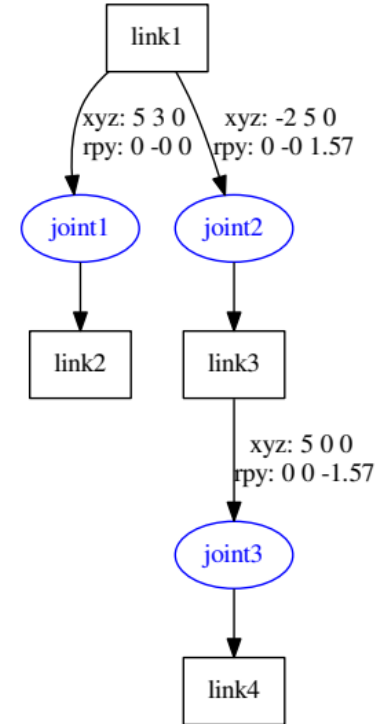
```
<robot name="test_robot">
  <link name="link1" />
  <link name="link2" />
  <link name="link3" />
  <link name="link4" />
  <joint name="joint1" type="continuous">
    <parent link="link1"/>
    <child link="link2"/>
    <origin xyz="5 3 0" rpy="0 0 0" />
    <axis xyz="-0.9 0.15 0" />
  </joint>
  <joint name="joint2" type="continuous">
    <parent link="link1"/>
    <child link="link3"/>
    <origin xyz="-2 5 0" rpy="0 0 1.57" />
    <axis xyz="-0.707 0.707 0" />
  </joint>
  <joint name="joint3" type="continuous">
    <parent link="link3"/>
    <child link="link4"/>
    <origin xyz="5 0 0" rpy="0 0 -1.57" />
    <axis xyz="0.707 -0.707 0" />
  </joint>
</robot>
```

Arbre de l'URDF

► Une visualisation d'un URDF sous forme d'arbre

- Utilisation du logiciel Graphviz pour afficher le graphe

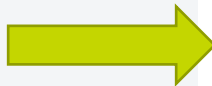
`urdf_to_Graphviz fichier.urdf`



Eviter les duplications en utilisant XACRO

Simplifie et permet de générer du code URDF

```
<link name="base_link">
  <visual>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
    <material name="blue"/>
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
  </collision>
</link>
```



```
<xacro:property name="width" value="0.2" />
<xacro:property name="bodylen" value="0.6" />
<link name="base_link">
  <visual>
    <geometry>
      <cylinder radius="${width}" length="${bodylen}" />
    </geometry>
    <material name="blue"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="${width}" length="${bodylen}" />
    </geometry>
  </collision>
</link>
```

XACRO

► Permet également

- Spécifier des constantes / variables

```
<xacro:property name="the_radius" value="2.1" />
<xacro:property name="the_length" value="4.5" />

<geometry type="cylinder" radius="${the_radius}" length="${the_length}" />
```

- Faire des fonctions mathématiques simples (additions, multiplications, divisions)

```
<xacro:property name="R" value="2" />
<xacro:property name="alpha" value="${30/180*pi}" />
<circle circumference="${2 * pi * R}" pos="${sin(alpha)} ${cos(alpha)}" />
<limit lower="${radians(-90)}" upper="${radians(90)}" effort="0" velocity="${radians(75)}" />
```

- Faire des opérations logiques (if, unless)

```
<xacro:if value="<expression>">
  <... some xml code here ...>
</xacro:if>
<xacro:unless value="<expression>">
  <... some xml code here ...>
</xacro:unless>
```

Les Macro Xacro

► Créer des blocs / fonctions

- Macro avec des paramètres
- Possibilité d'être appelé plusieurs fois
- Possibilité d'être importé dans d'autres fichiers

```
<xacro:include filename="$(find package)/other_file.xacro" />
<xacro:include filename="other_file.xacro" />
<xacro:include filename="$(pwd)/other_file.xacro" />
```

```
<xacro:macro name="leg" params="prefix reflect">
  <link name="${prefix}_leg">
    <visual>
      <geometry>
        <box size="${leglen} 0.1 0.2"/>
      </geometry>
      <origin xyz="0 0 -${leglen/2}" rpy="0 ${pi/2} 0"/>
      <material name="white"/>
    </visual>
    <collision>
      <geometry>
        <box size="${leglen} 0.1 0.2"/>
      </geometry>
      <origin xyz="0 0 -${leglen/2}" rpy="0 ${pi/2} 0"/>
    </collision>
    <xacro:default_inertial mass="10"/>
  </link>

  <joint name="base_to_${prefix}_leg" type="fixed">
    <parent link="base_link"/>
    <child link="${prefix}_leg"/>
    <origin xyz="0 ${reflect*(width+.02)} 0.25" />
  </joint>
  <!-- A bunch of stuff cut -->
</xacro:macro>
<xacro:leg prefix="right" reflect="1" />
<xacro:leg prefix="left" reflect="-1" />
```


Démonstration

URDF / Xacro

► Questions ?





Exercice 3

Remplacer un robot
dans un URDF

Correction



Manipulation avec ROS

MoveIt! 2

Qu'est-ce que MoveIt ?

► Un planificateur de trajectoire « On Line »

Un planificateur de trajectoire abstrait du robot:

- Prend en compte la cinématique du robot au chargement (URDF + SRDF)
- Gestion des autocollisions et des collisions avec l'environnement
- Possibilité de spécifier très précisément le problème (mais alors risque de ne pas trouver de solutions)

Intégration rapide avec les systèmes de contrôle sous ROS (ros_control):

- Peut superviser l'exécution de la trajectoire (si trop lent / trop d'écart, peut arrêter)
- Peut lancer les trajectoires avec les API moveit

Des outils:

- Génération automatique de la configuration
- Contrôle du robot « directe »

© 2010 Blackwell Publishing Ltd, *Journal of Internal Medicine* 267: 105–113



Les interfaces utilisateurs

► Permettent d'utiliser Movelt en restant sain d'esprit

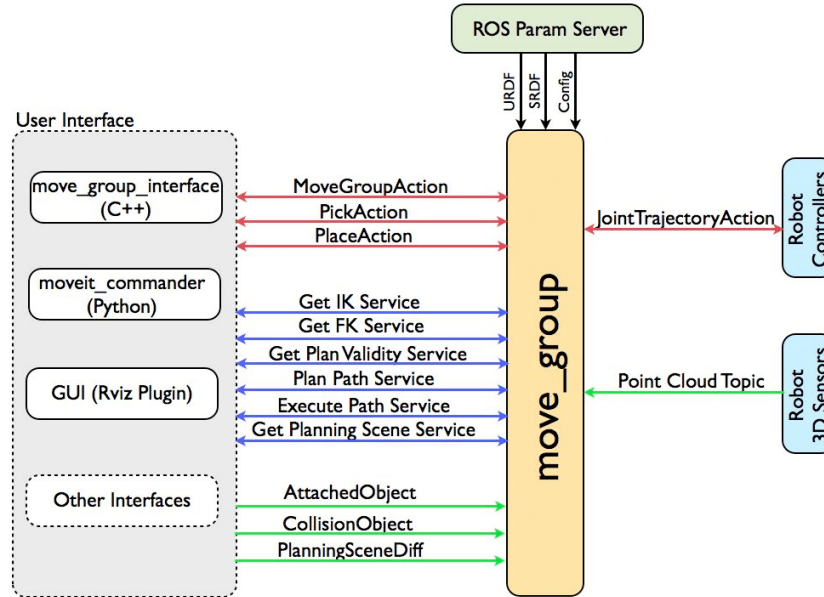
- Simplifie la création de trajectoire planifié
- Simplifie la modification de la scène (ajout d'objets, etc...)
- Possibilité de lancer la planification et l'exécution en 1 ligne
- **Interface déjà intégré avec Rviz (pratique pour tester, mais concrètement assez difficile à utiliser en situation réelle, peut être pratique pour modifier la scène).**



Démonstration

- **Interface RVIZ**
- **Planification**
- **Obstacle**
- **Exécution Virtuelle**

Le move group



Le move group

► Le Coeur de MoveIt!

- **Synchronise les actions**
 - Requête utilisateur ⇔ Planification ⇔ Scène Environment ⇔ Exécution
- **Synchronise les information robots/camera/capteurs (Robot Interface)**
 - Ecoute le topic /joint_state pour connaitre la position actuelle du robot (planification depuis la position actuelle, exécution de trajectoire,)
- **Configuration totale – Charge beaucoup de paramètres**
 - Utilise le paramètre robot_description (URDF)
 - Configuration des planificateurs / capteurs / contrôleur

Movelt2

► Questions ?





Manipulation partie 2

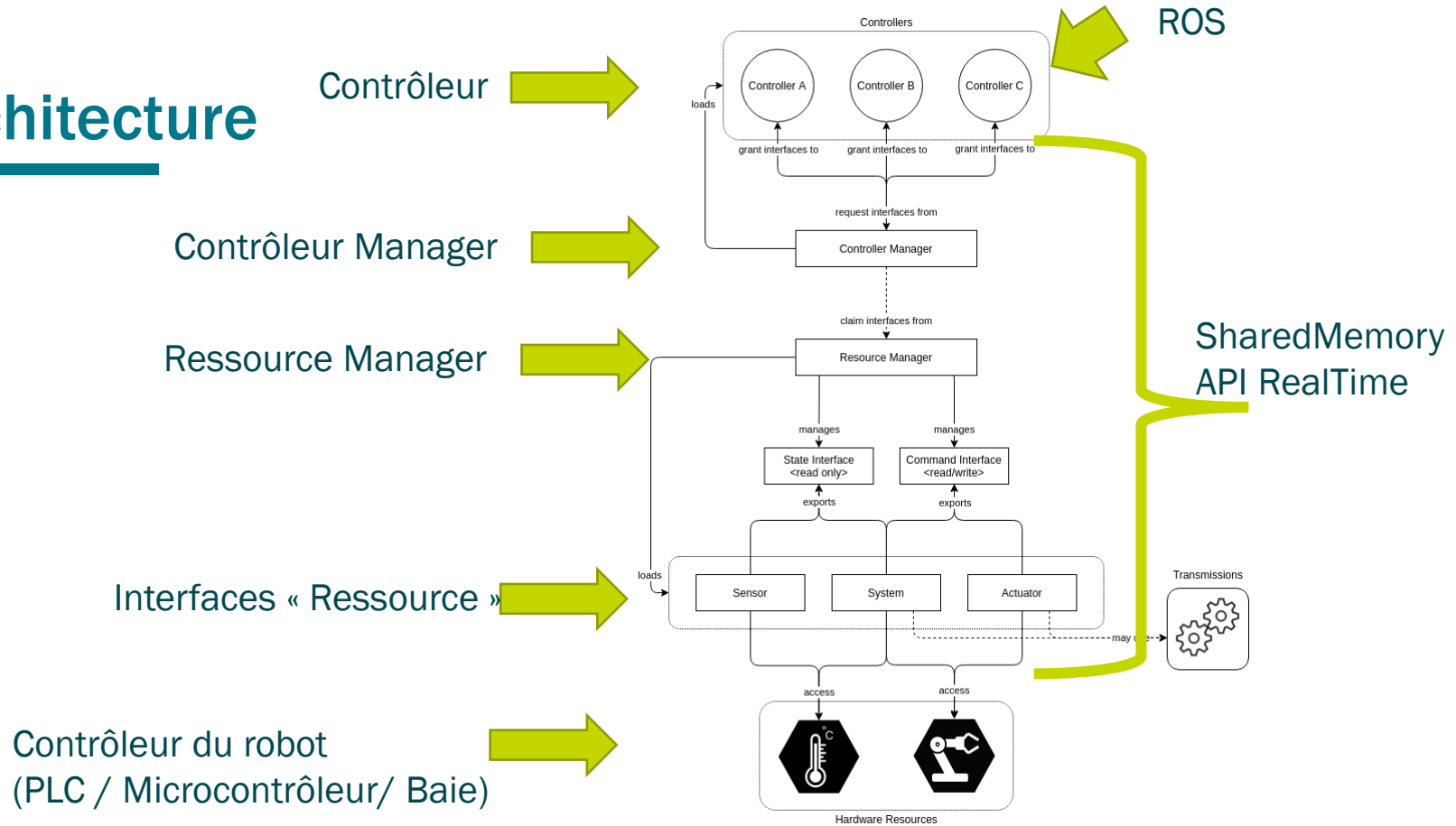
ROS 2 Control

Le package le plus “dur” de ROS

► Pourquoi ros2_control

- Fournir un niveau d'abstraction entre le hardware (spécifique) et les fonctions de commandes comme le suivi de trajectoire (indépendant)
- Fonctionne avec les concepts contrôleurs (et de broadcasters) et de ressources contrôlées
- Pas besoin de beaucoup de codes pour fonctionner, mais le code doit être précis et bien fait
- Contrôleur déjà prêt :
 - https://github.com/ros-controls/ros2_controllers
 - Position, Vitesse, Effort, Trajectoire, ...

Architecture



Le contrôleur manager

► Gère les contrôleurs et offres des interfaces pour en changer

- **Active, désactive, charge et décharges les contrôleurs (commande, load, unload, start, stop).**
- **Pour chaque contrôleur actifs et charger, => Les faits communiqués avec les ressources demandés**
 - Exécuteur de trajectoire en position => Une interface de contrôle en position sur chaque moteur listé.
- **Remonte les conflits (plusieurs contrôleurs demandent la même ressource)**
- **Utilise l'API implémenté dans les contrôleurs « update() »**

Le Resource Manager

► Gère les ressources hardware

- **Les ressources matérielles sont basées sur des templates:**
 - System – Multi-DOF – Organisme pour le contrôle
À utiliser quand on contrôle tout un robot depuis une commande (par exemple envoyer toutes les positions d'un coup a une baie robot)
 - Sensor – composant pour l'acquisition de donnée - Fourni seulement des données (aucun contrôle possible)
 - Actuator – 1 seul DOF – Contrôle simple d'un moteur
Peut fournir les données de capteurs
Doit fournir une interface de contrôle

Notation dans l'URDF

```
<ros2_control name="RRBotSystemPositionOnly" type="system">
  <hardware>
    <plugin>ros2_control_demo_hardware/RRBotSystemPositionOnlyHardware</plugin>
    <param name="example_param_write_for_sec">2</param>
    <param name="example_param_read_for_sec">2</param>
  </hardware>
  <joint name="joint1">
    <command_interface name="position">
      <param name="min">-1</param>
      <param name="max">1</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>
  <joint name="joint2">
    <command_interface name="position">
      <param name="min">-1</param>
      <param name="max">1</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>
</ros2_control>
```

```
<ros2_control name="RRBotGripper" type="actuator">
  <hardware>
    <plugin>ros2_control_demo_hardware/PositionActuatorHardware</plugin>
    <param name="example_param_write_for_sec">1.23</param>
    <param name="example_param_read_for_sec">3</param>
  </hardware>
  <joint name="gripper_joint ">
    <command_interface name="position">
      <param name="min">0</param>
      <param name="max">50</param>
    </command_interface>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>
</ros2_control>
```

Les Fakes Components

► Simulation d'un composant « parfait »

- Utilisé pour simuler le comportement d'un System
- Utilisez le plugin :

```
<plugin>fake_components/GenericSystem</plugin>  
<param name="fake_sensor_commands">${fake_sensor_commands}</param>  
<param name="state_following_offset">0.0</param>
```

- Il est possible d'avoir des paramètres pour créer des effets / défauts (délais sur la commande etc...)

ROS 2 Control

► Questions ?





Exercice 4

Déplacer un robot
avec RVIZ

Configurer un robot
avec MoveIt!

Correction

- Avez-vous des questions ?

Bilan

► Vu aujourd'hui

- Les bases de ROS
- Les concepts clefs
- Les outils principaux
- Les bases de la manipulation avec ROS
- Les outils pour la manipulation avec ROS

ROS 2

► Questions ?





LE FUTUR
DE VOS USINES

Contact

Ludovic DELVAL

ludovic.delval@irt-jules-verne.fr

LinkedIN : <https://www.linkedin.com/in/ludovic-delval-ros>

IRT JULES VERNE

Chemin du Chaffault,
44 340 Bouguenais, France
contact@irt-jules-verne.fr

WWW.IRT-JULES-VERNE.FR

Rejoignez-nous sur :

